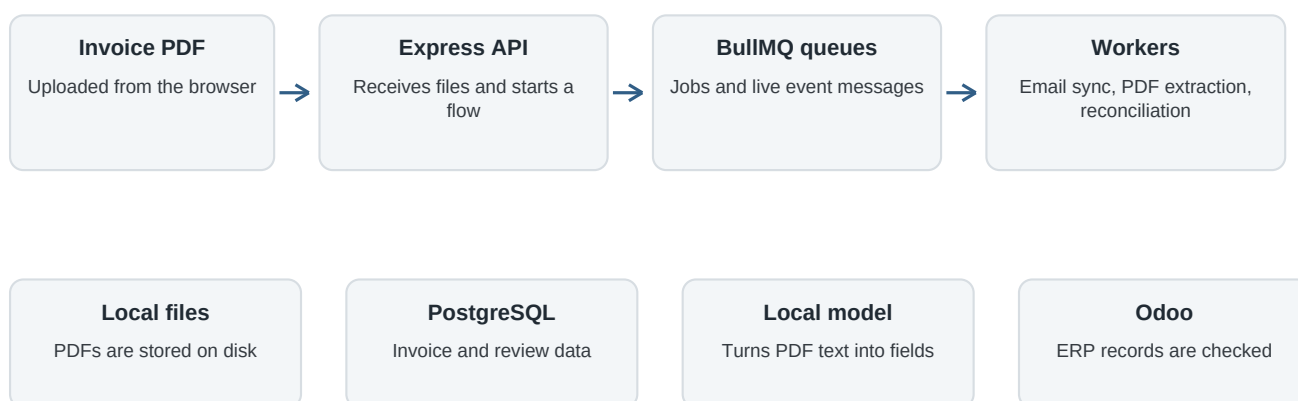


ClearPo Invoice Processing

I built ClearPo to help review invoices from the time a PDF is uploaded until a decision is made. The back end is written in TypeScript with Express. It uses background workers for the longer tasks, so file parsing and invoice checks do not have to happen inside the upload request.

Architecture



The workers use these services as each invoice moves through the flow.

The backend also sends live progress to the front end through server sent events.

How the parts fit together

The API receives the PDF and adds a job to the queue. Workers pick up those jobs, save invoice information in PostgreSQL, and use the PDF parser, local language model, and Odoo when needed. The front end gets invoice details from the API and follows progress through a live event stream.

From upload to extracted data

This is the part of the flow that prepares an invoice for reconciliation.

01 Upload the PDF

The Express upload endpoint accepts a PDF. Right now I use Multer to save the file in the back end's local uploads folder. The upload and its file details are then put on a BullMQ queue.

02 Save the invoice record

The email sync worker picks up the queued file details and creates the initial invoice record in PostgreSQL. This is what lets the front end show a new invoice before extraction has finished.

03 Start the flow

When processing is started, the API gets the invoice record from the database and adds a PDF extraction job to the queue. The worker gets the file location and reads the PDF.

04 Read and extract the PDF

The worker passes the file through a PDF parser to get its text. It sends that text to a small language model running locally and asks for structured invoice fields such as supplier name, invoice number, amount, and purchase order.

05 Save the extracted fields

When the model returns the fields, the worker saves them to PostgreSQL and publishes them to the front end. It also adds the extracted invoice to the reconciliation queue.

Model choice

I chose a small local model to keep the project inexpensive to run and to avoid sending invoice text to an external model service. The current configuration uses a quantized Llama 3.2 3B Instruct model.

How the final decision is made

Once extraction is complete, the reconciliation worker compares the invoice with the purchase order and other records in Odoo.

01 Deterministic checks run first

The worker checks the invoice against the purchase order, supplier, amounts, line items, payment state, and receipt information. These checks always run. When the result is clear, the system can continue without calling the agent.

02 The agent investigates unclear cases

If something is ambiguous, such as a supplier mismatch or a possible duplicate purchase order, the agent can investigate. I connected it to read only Odoo tools so it can look up ERP records but cannot change them or make a payment.

03 The policy engine returns a result

The policy engine uses the reconciliation findings and, when needed, the agent's findings to return a decision and a reason. Depending on the checks, the result can be ready for payment, require review, or be blocked.

04 The front end follows the flow

The front end loads invoice records from the API and listens for server sent events for the selected invoice. Status updates move the journey timeline forward. Extracted data fills in the invoice details, and the final event includes the decision and its reason.

Current status

PDFs are stored on local disk in the current version. For production, I plan to move them to blob storage and save the resulting file location with the invoice metadata. Odoo is the ERP connected to the reconciliation flow. Slack integration is still in development.